

EDUGENIE

Phase 2 – Requirement Analysis

AI-Powered Learning Assistant

1. Introduction

This document presents the requirement analysis for EduGenie, an AI-powered learning assistant proposed to support students with educational question answering, topic explanation, text summarization, quiz generation, and learning recommendations. The requirements define the expected behavior, system qualities, users, inputs, outputs, and technology direction of the application.

2. Project Goal

The goal of EduGenie is to provide a simple web-based learning assistant that accepts student input, uses Generative AI to process learning-related requests, and presents useful responses through an accessible interface.

3. Target Users

- Students who need help understanding educational concepts.
- Students who want quick answers to academic questions.
- Students who want to summarize study material.
- Students who want to practice using automatically generated quizzes.
- Learners who want recommendations for what to study next.

4. Functional Requirements

ID	Requirement	Expected Behavior
FR-01	Question & Answer	The system shall accept an educational question and return an AI-generated answer.
FR-02	Topic Explanation	The system shall accept a topic and provide a simplified explanation.
FR-03	Text Summarization	The system shall accept educational text and return a concise summary.
FR-04	Quiz Generation	The system shall accept a topic or learning input and generate multiple-choice questions.
FR-05	Learning Recommendations	The system shall accept a learning topic and provide structured recommendations.

FR-06	Web Interface	The system shall provide a browser-based interface through which users can access the learning features.
FR-07	AI Processing	The system shall send appropriate requests to the configured Generative AI service and process the response.
FR-08	Response Display	The system shall display the generated result to the user in the web interface.

5. Detailed Feature Requirements

5.1 Question & Answer

- User provides a question in natural language.
- The backend receives the request.
- The AI service generates an educational response.
- The generated response is returned and displayed to the user.

5.2 Topic Explanation

- User provides a topic.
- The system requests a clear explanation from the AI service.
- The response should be understandable and suitable for learning.

5.3 Text Summarization

- User provides educational text.
- The system sends the content for summarization.
- The application displays a concise summary.

5.4 Quiz Generation

- User provides a topic or relevant learning input.
- The AI service generates practice multiple-choice questions.
- Questions include answer options and a correct answer.
- The frontend displays the generated quiz.

5.5 Learning Recommendations

- User enters a topic they want to learn.
- The system requests a structured learning path from the AI service.
- The response provides useful topics or study recommendations.

6. Non-Functional Requirements

Category	Requirement	Description
----------	-------------	-------------

Usability	Simple interface	The application should be easy for students to understand and operate.
Performance	Responsive requests	The application should process requests and display results without unnecessary delay, subject to AI service response time.
Reliability	Error handling	The application should handle invalid requests or service failures gracefully.
Security	API key protection	Sensitive credentials should be stored in environment variables and not exposed in source control.
Maintainability	Modular code	Features should be separated into appropriate backend modules.
Scalability	Extensible design	The application structure should allow additional learning features to be added later.
Compatibility	Web browser access	The application should be usable through a modern web browser.
Testability	Automated testing	Core API behavior should be testable using an automated testing framework.

7. Hardware Requirements

- Computer or laptop capable of running Python and a modern web browser.
- Internet connection for communication with the Generative AI service.
- Sufficient storage for the project files, Python environment, and dependencies.

8. Software Requirements

- Python 3.11 or later.
- Visual Studio Code or another suitable code editor.
- FastAPI and Uvicorn for the web backend.
- Google Gemini / Google GenAI service for Generative AI.
- HTML, CSS, and JavaScript for the frontend.
- Pydantic for data validation.
- python-dotenv for environment configuration.
- Pytest and HTTPX for application testing.
- Git and GitHub for version control and project submission.

9. Input Requirements

Depending on the selected feature, the system may accept:

- A natural-language educational question.
- A topic name for explanation.
- Educational text for summarization.
- A topic or learning subject for quiz generation.
- A topic for learning recommendations.

10. Output Requirements

The application should return an appropriate AI-generated result, such as:

- An answer to the submitted question.
- A clear explanation of the selected topic.
- A concise summary of submitted text.
- A generated multiple-choice quiz.
- A structured learning recommendation or learning path.

11. AI Integration Requirements

- The application shall use a configured Generative AI model.
- The API key shall be supplied through an environment variable.
- AI requests shall be handled by a dedicated backend client/module.
- The application should provide suitable error handling when the AI service cannot be reached or returns an unusable response.
- AI-generated content should be treated as assistance and may require verification for academic accuracy.

12. Security Requirements

- The real API key must not be committed to the public GitHub repository.
- The .env file should be excluded through .gitignore.
- A .env.example file may be provided as a configuration template without real credentials.
- User input should be validated before being processed.
- Errors should not expose sensitive configuration values.

13. Constraints and Assumptions

- AI features depend on availability and configuration of the selected Generative AI service.
- Internet access is required for live AI requests.
- AI-generated responses may vary between requests.
- The initial version is intended as an educational project prototype.
- The initial requirements do not assume persistent user accounts or long-term learning history.

14. Acceptance Criteria

- The application can be started successfully using the documented setup process.
- A user can access the web interface through a browser.

- The Q&A feature accepts a question and displays a generated response.
- The explanation feature accepts a topic and displays an explanation.
- The summarization feature accepts text and displays a summary.
- The quiz feature generates a usable multiple-choice quiz.
- The learning recommendation feature returns a structured learning response.
- Sensitive API credentials are not included in the public repository.
- Core API behavior can be tested using the project's test suite.

15. Summary

The requirement analysis establishes the functional and non-functional requirements for EduGenie. The system is planned as a web-based Generative AI learning assistant with five primary learning features: Q&A, topic explanation, text summarization, quiz generation, and learning recommendations. These requirements provide the basis for the Project Design and Project Planning phases.